Runge–Kutta Track Extrapolation in the LHCb Software Stack

Mathematical Foundations, Numerical Schemes, and the QMTest Workflow

Auto-generated Technical Reference

March 2026

Contents

1 Introduction **3**

2 State Vector and Conventions **3**

3 Equations of Motion **3**

4 General Adaptive Runge–Kutta Framework **4**
- 4.1 Embedded RK Pairs . . . 4
- 4.2 FSAL Property . . . 4

5 Cash–Karp Method (Default) **5**
- 5.1 Butcher Tableau . . . 5
- 5.2 Justification . . . 5

6 Other Embedded RK Schemes **5**
- 6.1 Dormand–Prince 5(4) . . . 5
- 6.2 Bogacki–Shampine 3(2) . . . 6
- 6.3 Verner Family (6th–9th Order) . . . 6

7 Adaptive Step-Size Control **6**
- 7.1 Acceptance Criterion . . . 6
- 7.2 Step Scaling . . . 6
- 7.3 Termination Conditions . . . 6

8 Analytic Jacobian Propagation **7**
- 8.1 Derivatives of the Force Function . . . 7
- 8.2 Variational Equation . . . 7
- 8.3 Numerical Jacobian Mode . . . 7

9 Parabolic Extrapolator **7**
- 9.1 Algorithm . . . 8
- 9.2 Transport Matrix . . . 8
- 9.3 Justification . . . 8

10 STEP Extrapolator (Runge–Kutta–Nyström) **8**
- 10.1 Nyström Structure . . . 8
- 10.2 Stage Evaluations . . . 8
- 10.3 Update Formulae . . . 9
- 10.4 Error Estimate . . . 9

10.5 Advantages . . . 9

11 Herab Extrapolator **9**
11.1 Step Control . . . 9

12 Kisel Extrapolator **9**
12.1 Approach . . . 9
12.2 Justification . . . 10

13 Linear Extrapolator **10**

14 Parametrized Extrapolator **10**
14.1 Training . . . 10

15 Master Extrapolator **10**
15.1 Dispatch Logic . . . 10
15.2 Justification . . . 11
15.3 Stepping and Material . . . 11

16 QMTest Regression Testing Pipeline **11**
16.1 Test State Grid . . . 12
16.2 Extrapolators Under Test . . . 12
16.3 Validation . . . 12

17 SOA Benchmark Tester **13**

18 Summary Table of Extrapolators **13**

1 Introduction

Track extrapolation is the process of propagating a charged particle's state through the LHCb detector from one z-plane to another, accounting for the deflection due to the dipole magnetic field and (optionally) energy loss and multiple scattering in detector material. The extrapolation engine solves the equations of motion for a charged particle in a magnetic field numerically, using several Runge–Kutta (RK) variants.

This document provides:

1. The equations of motion being solved (Section 3).
2. The state vector and transport matrix conventions (Section 2).
3. Mathematical derivations for every RK scheme implemented: Cash–Karp, Dormand–Prince, Bogacki–Shampine, Verner, STEP/RKN, Herab, Kisel, and the parabolic single-step approximation (Sections 4–9).
4. The adaptive step-size control algorithm (Section 7).
5. The analytic Jacobian propagation (Section 8).
6. The `TrackMasterExtrapolator` dispatch logic (Section 15).
7. The QMTest regression testing pipeline (Section 16).

2 State Vector and Conventions

All LHCb extrapolators represent the track state at a given z-position as a five-component vector:

$$\mathbf{s} = \begin{pmatrix} x \\ y \\ t_x \\ t_y \\ q/p \end{pmatrix} \quad \text{at position } z, \tag{1}$$

where x, y are the transverse positions, $t_x = dx/dz$ and $t_y = dy/dz$ are the direction tangents, and q/p is the charge divided by the total momentum. The independent variable is z (the beam axis), not time.

The **transport matrix** (Jacobian) is a 5×5 matrix F satisfying

$$\mathbf{s}_{\text{out}} \approx F \cdot \mathbf{s}_{\text{in}}, \tag{2}$$

used to propagate the covariance matrix via the similarity transformation

$$C_{\text{out}} = F\, C_{\text{in}}\, F^T. \tag{3}$$

3 Equations of Motion

A charged particle with charge q and momentum p in a magnetic field $\mathbf{B} = (B_x, B_y, B_z)$ obeys the Lorentz force law. In the (x, y, t_x, t_y) representation with z as the independent variable, the ODE is:

$$\frac{d}{dz}\begin{pmatrix} x \\ y \\ t_x \\ t_y \end{pmatrix} = \begin{pmatrix} t_x \\ t_y \\ (q/p)\, c_{\text{light}}\, \mathcal{A}_x \\ (q/p)\, c_{\text{light}}\, \mathcal{A}_y \end{pmatrix}, \tag{4}$$

where

$$n = \sqrt{1 + t_x^2 + t_y^2}, \tag{5}$$

$$\mathcal{A}_x = n\big[\, t_y(t_x B_x + B_z) - (1 + t_x^2)\, B_y \,\big], \tag{6}$$

$$\mathcal{A}_y = n\big[\, -t_x(t_y B_y + B_z) + (1 + t_y^2)\, B_x \,\big], \tag{7}$$

and $c_{\text{light}} = 299.792458$ mm/ns converts from natural units (Gaudi convention).

Physical justification. The terms $\mathcal{A}_x$ and $\mathcal{A}_y$ arise from projecting the three-dimensional Lorentz force $\mathbf{F} = q\,\mathbf{v} \times \mathbf{B}$ onto the (x, y) plane using the chain rule $dt_x/dz = (dt_x/dt)(dt/dz)$. The factor n accounts for the arc-length correction $ds/dz = n$, and the q/p factor appears because the Lorentz force changes direction but not speed, giving the curvature $\kappa \propto qB/p$.

4 General Adaptive Runge–Kutta Framework

4.1 Embedded RK Pairs

An *embedded* Runge–Kutta method of order $p(p-1)$ computes two estimates of the solution at each step: a higher-order estimate $\mathbf{y}_{n+1}$ used to advance the solution, and a lower-order estimate $\hat{\mathbf{y}}_{n+1}$ used solely for error control. The method is defined by its **Butcher tableau**:

$$\begin{array}{c|cccc}
c_1 & & & & \\
c_2 & a_{21} & & & \\
\vdots & \vdots & \ddots & & \\
c_s & a_{s1} & \cdots & a_{s,s-1} & \\
\hline
 & b_1 & b_2 & \cdots & b_s \\
 & b_1^* & b_2^* & \cdots & b_s^*
\end{array} \tag{8}$$

The stage values are computed as:

$$\mathbf{k}_i = h\,\mathbf{f}\left(z_n + c_i h,\ \mathbf{y}_n + \sum_{j=1}^{i-1} a_{ij}\,\mathbf{k}_j \right), \qquad i = 1, \ldots, s, \tag{9}$$

and the two estimates are:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \sum_{i=1}^{s} b_i\,\mathbf{k}_i \quad (\text{order } p), \tag{10}$$

$$\hat{\mathbf{y}}_{n+1} = \mathbf{y}_n + \sum_{i=1}^{s} b_i^*\,\mathbf{k}_i \quad (\text{order } p-1). \tag{11}$$

The local truncation error estimate is:

$$\mathbf{e} = \mathbf{y}_{n+1} - \hat{\mathbf{y}}_{n+1} = \sum_{i=1}^{s} (b_i - b_i^*)\,\mathbf{k}_i. \tag{12}$$

4.2 FSAL Property

Some schemes (Dormand–Prince, Tsitouras) satisfy the *First Same As Last* (FSAL) property: the last stage $\mathbf{k}_s$ of step n equals the first stage $\mathbf{k}_1$ of step $n+1$. This saves one function evaluation per accepted step.

5 Cash–Karp Method (Default)

The Cash–Karp method is a 6-stage, 5th-order embedded pair with a 4th-order error estimator. It is the default scheme in `TrackRungeKuttaExtrapolator`.

5.1 Butcher Tableau

$$
\begin{array}{c|cccccc}
0 & & & & & & \\
\frac{1}{5} & \frac{1}{5} & & & & & \\
\frac{3}{10} & \frac{3}{40} & \frac{9}{40} & & & & \\
\frac{3}{5} & \frac{3}{10} & -\frac{9}{10} & \frac{6}{5} & & & \\
1 & -\frac{11}{54} & \frac{5}{2} & -\frac{70}{27} & \frac{35}{27} & & \\
\frac{7}{8} & \frac{1631}{55296} & \frac{175}{512} & \frac{575}{13824} & \frac{44275}{110592} & \frac{253}{4096} & \\
\hline
b_i & \frac{37}{378} & 0 & \frac{250}{621} & \frac{125}{594} & 0 & \frac{512}{1771} \\
b_i^* & \frac{2825}{27648} & 0 & \frac{18575}{48384} & \frac{13525}{55296} & \frac{277}{14336} & \frac{1}{4}
\end{array}
\tag{13}
$$

5.2 Justification

Cash and Karp (1990) designed this pair to minimize the leading-order truncation error coefficient of the 5th-order formula while simultaneously providing an efficient 4th-order embedded estimator. The choice of $c_6 = 7/8$ (rather than 1) places the final stage evaluation interior to the step, providing better error estimation for stiff-like problems. Compared to the Fehlberg 4(5) pair, Cash–Karp gives more reliable step-size estimates and is preferred for non-stiff systems such as charged-particle tracking.

6 Other Embedded RK Schemes

The `TrackRungeKuttaExtrapolator` supports 15 additional schemes:

Table 1: Available embedded RK schemes in the LHCb extrapolator.

Scheme	**Stages**	**Order**	**FSAL**	**Reference**
Cash–Karp (default)	6	5(4)	No	Cash & Karp, 1990
Fehlberg	6	5(4)	No	Fehlberg, 1969
Dormand–Prince	7	5(4)	Yes	Dormand & Prince, 1980
Bogacki–Shampine 3	4	3(2)	No	Bogacki & Shampine, 1989
Bogacki–Shampine 5	8	5(4)	No	Bogacki & Shampine
Heun–Euler	2	2(1)	No	Classical
Sharp–Smart 5	7	5(4)	No	Sharp & Smart
Tsitouras 5	7	5(4)	Yes	Tsitouras, TEI Chalkis
Tsitouras 2009	7	5(4)	No	AIP Conf. Proc. 1168
Sharp–Verner 6	9	6(5)	Yes	SIAM J. Numer. Anal. 31
Sharp–Verner 7	12	7(6)	Yes	SIAM J. Numer. Anal. 31
Verner 6	8	6(5)	No	Verner
Verner 7	10	7(6)	No	Verner
Verner 8	13	8(7)	No	Verner
Verner 9	16	9(8)	No	Verner

6.1 Dormand–Prince 5(4)

This is the most widely used adaptive RK scheme (MATLAB's `ode45`). It satisfies FSAL, requiring only 6 function evaluations per accepted step. The Butcher coefficients are optimised to minimise

the local truncation error norm:

$$c = \left(0,\ \tfrac{1}{5},\ \tfrac{3}{10},\ \tfrac{4}{5},\ \tfrac{8}{9},\ 1,\ 1\right).$$

6.2 Bogacki–Shampine 3(2)

A low-order ($p = 3$) pair with only 4 stages, useful when very high accuracy is not needed but computational cost must be minimised. Each step requires only 3 function evaluations (with FSAL). It is used in the LHCb test suite for cross-validation.

6.3 Verner Family (6th–9th Order)

Higher-order methods by Jim Verner provide greater accuracy per step at the cost of more stages. The 8th-order Verner scheme with 13 stages can take much larger steps for the same tolerance, which can be advantageous for long propagations through weak-field regions.

7 Adaptive Step-Size Control

Given the error estimate $\mathbf{e}$ from (12), the step is accepted or rejected based on the normalised error:

$$E = \max\left(\frac{|e_x|}{\epsilon_x},\ \frac{|e_y|}{\epsilon_y},\ \frac{|e_{t_x}|}{\epsilon_{t_x} + |\Delta t_x|\,\epsilon_{\mathrm{rel}}}\right), \tag{14}$$

where $\epsilon_x = \epsilon_y =$ `Tolerance` (default 0.001 mm) and $\epsilon_{\mathrm{rel}} =$ `RelToleranceTx` (default 5×10^{-5}). The mixed absolute–relative criterion for t_x prevents overly small steps when the curvature (hence Δt_x) is large.

7.1 Acceptance Criterion

The step is **accepted** if $E \le \sigma$ (default $\sigma = 5.5$) or if the step is already at the minimum step size $h_{\min}$ (default 10 mm). The generous threshold $\sigma > 1$ prioritises speed over strict accuracy, relying on error cancellation across multiple steps.

7.2 Step Scaling

The new step size is computed via the standard PI controller:

$$h_{n+1} = h_n \cdot \mathrm{clamp}(f,\ f_{\min},\ f_{\max}), \qquad f = \begin{cases} \max\!\left(f_{\min},\ S \cdot E^{-1/4}\right) & \text{if } E > 1 \text{ (shrink)}, \\ \min\!\left(f_{\max},\ S \cdot E^{-1/5}\right) & \text{if } 0 < E \le 1 \text{ (grow)}, \\ f_{\max} & \text{if } E = 0, \end{cases} \tag{15}$$

with safety factor $S = 1.0$, $f_{\min} = 0.125$, $f_{\max} = 4.0$.

Mathematical basis. For a method of order p, the local error scales as $\mathcal{O}(h^{p+1})$, so to achieve a target error ϵ from a current error $E\epsilon$, the optimal scaling is $h_{\mathrm{new}}/h_{\mathrm{old}} = (1/E)^{1/(p+1)}$. The shrink exponent $-1/4$ and grow exponent $-1/5$ correspond to the 4th- and 5th-order error estimates respectively, following the prescription of Numerical Recipes.

7.3 Termination Conditions

The propagation terminates with failure if any of these hold:

- Curling: $\max(|t_x|, |t_y|) > 10$ (`MaxSlope`).
- Excessive curvature: $|(q/p)\,B_y| > 1/\mathrm{m}$ (`MaxCurvature`).
- Step count: steps ≥ 1000 (`MaxNumSteps`).
- Out of bounds: $|x|, |y|, |z| > 100$ m.

8 Analytic Jacobian Propagation

The 5×5 transport matrix is computed by solving the **variational equations** alongside the trajectory. Since q/p is constant (no energy loss in the RK step), the Jacobian has the structure:

$$F = \begin{pmatrix} 1 & 0 & J_{x,t_x} & J_{x,t_y} & J_{x,q/p}c \\ 0 & 1 & J_{y,t_x} & J_{y,t_y} & J_{y,q/p}c \\ 0 & 0 & J_{t_x,t_x} & J_{t_x,t_y} & J_{t_x,q/p}c \\ 0 & 0 & J_{t_y,t_x} & J_{t_y,t_y} & J_{t_y,q/p}c \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \tag{16}$$

where $c \equiv c_{\text{light}}$.

8.1 Derivatives of the Force Function

The variational equations require the partial derivatives of $\mathcal{A}_x$ and $\mathcal{A}_y$ with respect to t_x and t_y:

$$\frac{\partial \mathcal{A}_x}{\partial t_x} = \frac{t_x \mathcal{A}_x}{n^2} + n(t_y B_x - 2t_x B_y), \tag{17}$$

$$\frac{\partial \mathcal{A}_x}{\partial t_y} = \frac{t_y \mathcal{A}_x}{n^2} + n(t_x B_x + B_z), \tag{18}$$

$$\frac{\partial \mathcal{A}_y}{\partial t_x} = \frac{t_x \mathcal{A}_y}{n^2} + n(-t_y B_y - B_z), \tag{19}$$

$$\frac{\partial \mathcal{A}_y}{\partial t_y} = \frac{t_y \mathcal{A}_y}{n^2} + n(-t_x B_y + 2t_y B_x). \tag{20}$$

8.2 Variational Equation

The 4×3 Jacobian sub-matrix J (derivatives of (x, y, t_x, t_y) w.r.t. $(t_{x,0}, t_{y,0}, q/p \cdot c_{\text{light}})$) evolves as:

$$\frac{d}{dz}\begin{pmatrix} J_{t_x} \\ J_{t_y} \end{pmatrix} = \frac{q}{p} c \begin{pmatrix} \partial_{t_x}\mathcal{A}_x & \partial_{t_y}\mathcal{A}_x \\ \partial_{t_x}\mathcal{A}_y & \partial_{t_y}\mathcal{A}_y \end{pmatrix} \begin{pmatrix} J_{t_x} \\ J_{t_y} \end{pmatrix}, \tag{21}$$

with $J_x = \int J_{t_x}\, dz$ and similarly for y. The q/p column has an additional inhomogeneous term: $\mathcal{A}_x$ itself contributes as the direct derivative $\partial(q/p\, c_{\text{light}}\, \mathcal{A}_x)/\partial(q/p\, c_{\text{light}}) = \mathcal{A}_x$.

8.3 Numerical Jacobian Mode

As a validation tool, the analytic Jacobian can be replaced by finite differences (`NumericalJacobian=true`):

$$J_{ij} \approx \frac{s_i(\mathbf{s}_0 + \delta_j\, \mathbf{e}_j) - s_i(\mathbf{s}_0)}{\delta_j}, \tag{22}$$

using perturbations $\delta_{t_x} = \delta_{t_y} = 0.01$, $\delta_{q/p} = 10^{-8}$. The same step sequence from the reference propagation is replayed for each perturbed state.

9 Parabolic Extrapolator

The `TrackParabolicExtrapolator` provides a fast, single-step approximation valid for short propagation distances where the field can be treated as approximately constant.

9.1 Algorithm

1. Evaluate the field at the **midpoint**: $\mathbf{B} = \mathbf{B}(x + \frac{1}{2}\, t_x \Delta z,\ y + \frac{1}{2}\, t_y \Delta z,\ z + \frac{1}{2}\Delta z)$.
2. Compute $\mathcal{A}_x$, $\mathcal{A}_y$ from (6)–(7).
3. Define $f = e\, c_{\text{light}}\, \Delta z\, (q/p)$, where e is the elementary charge.
4. Update slopes and positions:

$$t'_x = t_x + \mathcal{A}_x \cdot f, \tag{23}$$
$$x' = x + \Delta z\, t_x + \tfrac{1}{2}\, \mathcal{A}_x \cdot f \cdot \Delta z. \tag{24}$$

(Analogous for y, t_y.)

9.2 Transport Matrix

$$F_{\text{parab}} = \begin{pmatrix} 1 & 0 & \Delta z + \frac{1}{2}\frac{\partial \mathcal{A}_x}{\partial t_x} f \Delta z & \frac{1}{2}\frac{\partial \mathcal{A}_x}{\partial t_y} f \Delta z & \frac{1}{2}\mathcal{A}_x e c \Delta z^2 \\ 0 & 1 & \frac{1}{2}\frac{\partial \mathcal{A}_y}{\partial t_x} f \Delta z & \Delta z + \frac{1}{2}\frac{\partial \mathcal{A}_y}{\partial t_y} f \Delta z & \frac{1}{2}\mathcal{A}_y e c \Delta z^2 \\ 0 & 0 & 1 + \frac{\partial \mathcal{A}_x}{\partial t_x} f & \frac{\partial \mathcal{A}_x}{\partial t_y} f & \mathcal{A}_x e c \Delta z \\ 0 & 0 & \frac{\partial \mathcal{A}_y}{\partial t_x} f & 1 + \frac{\partial \mathcal{A}_y}{\partial t_y} f & \mathcal{A}_y e c \Delta z \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \tag{25}$$

with the partial derivatives given by (17)–(20).

9.3 Justification

The parabolic approximation treats the trajectory as a **parabola** (constant curvature) over each step. This is equivalent to a 1-stage midpoint method: error $\mathcal{O}(\Delta z^3)$. For $|\Delta z| < 100$ mm (the threshold used by `TrackMasterExtrapolator`), the field changes negligibly, so the single midpoint evaluation suffices. The speed advantage is substantial: one field lookup vs. six for Cash–Karp.

10 STEP Extrapolator (Runge–Kutta–Nyström)

The `TrackSTEPExtrapolator` implements the ATLAS STEP algorithm (Lund, Bugge, Gavrilenko, Strandlie), which exploits the second-order structure of the ODE.

10.1 Nyström Structure

The equations of motion (4) have the structure of a *second-order* ODE for positions:

$$\frac{d^2x}{dz^2} = \frac{dt_x}{dz} = (q/p)\, c_{\text{light}}\, \mathcal{A}_x(t_x, t_y, \mathbf{B}),$$

where $t_x = dx/dz$. The Runge–Kutta–Nyström (RKN) methods directly integrate this second-order form, updating both x and t_x with separate formulae that take advantage of the double integration.

10.2 Stage Evaluations

$$\text{Stage 0:} \quad z_0,\ \mathbf{B}_0,\ k_0 = f(t_x, t_y, \mathbf{B}_0), \tag{26}$$

$$\text{Stage 1:} \quad z_1 = z_0 + h/2, \tag{27}$$

$$x_1 = x_0 + \tfrac{h}{2}\, t_x + \tfrac{h^2}{8}\, k_0, \quad t_{x,1} = t_x + \tfrac{h}{2}\, k_0, \tag{28}$$

$$\text{Stage 2:} \quad \text{same } z_1,\ x_1 \text{ (reuse } \mathbf{B}_1), \quad t_{x,2} = t_x + \tfrac{h}{2}\, k_1, \tag{29}$$

$$\text{Stage 3:} \quad z_3 = z_0 + h, \quad x_3 = x_0 + h\big(t_x + \tfrac{h}{2}\, k_2\big), \quad t_{x,3} = t_x + h\, k_2. \tag{30}$$

10.3 Update Formulae

$$x_{n+1} = x_n + h\,t_x + \frac{h^2}{6}(k_0 + k_1 + k_2), \tag{31}$$

$$t_{x,n+1} = t_x + \frac{h}{6}(k_0 + 2k_1 + 2k_2 + k_3). \tag{32}$$

10.4 Error Estimate

$$\mathrm{err} = h^2\,(k_0 - k_1 - k_2 + k_3). \tag{33}$$

10.5 Advantages

1. Only **4 stages** with **2 field evaluations** (stages 1 and 2 share the same position).
2. The endpoint field (stage 3) can be **reused** as stage 0 of the next step (`UseFieldLastStep`).
3. For track extrapolation where position accuracy matters more than slope accuracy, the Nyström form provides one extra order for positions.

11 Herab Extrapolator

The `TrackHerabExtrapolator` (A. Spiridonov, HERA-B, 1998) provides five sub-methods selectable via the `extrapolatorID` property.

Table 2: Herab extrapolator sub-methods.

ID	Name	RK Order	Jacobian
1	`rk4fast`	4	Simplified ($d/d(q/p)$ only)
2	`rk4order`	4	Full analytic
3	`rk5fast`	5 (Cash–Karp)	Simplified
4	`rk5order`	5 (Cash–Karp)	Full analytic
5	`rk5numde`	5 (Cash–Karp)	Numerical differentiation

The default method (ID 5, `rk5numde`) uses Cash–Karp for the trajectory and computes the Jacobian by finite differences: perturbing each of the 5 state components by a small amount, re-propagating, and computing $J_{ij} = \Delta s_i/\delta_j$.

11.1 Step Control

Unlike the main RK extrapolator's continuous scaling, the Herab extrapolator uses **binary halving**: if $|x_{5\mathrm{th}} - x_{4\mathrm{th}}| > \epsilon$, the step is split in half. If it still fails and $|h| > h_{\min}$, it falls back to two 4th-order (RK4) half-steps.

12 Kisel Extrapolator

The `TrackKiselExtrapolator` (I. Kisel, CBM experiment) uses a **Taylor expansion** of the exact solution in powers of the field integral.

12.1 Approach

Define $h = (q/p)\,c_{\mathrm{light}}\,n$ where $n = \sqrt{1 + t_x^2 + t_y^2}$. The solution is expanded as:

$$t'_x = t_x + h\,n\,[s_{\mathcal{A}_1} + h\,n\,(s_{\mathcal{A}_2} + h\,n\,s_{\mathcal{A}_3})]\,, \tag{34}$$

where $s_{\mathcal{A}_k}$ are *field integrals* computed via Simpson-like quadrature at 3 points (start, midpoint, end):

$$s_i = \frac{\Delta z}{6}(B_i^{(0)} + 4B_i^{(1)} + B_i^{(2)}). \tag{35}$$

The configurable `order` property (1, 2, or 3) controls how many terms in the Taylor expansion are retained. Order 1 gives a 2-point approximation, order 3 includes rank-3 tensor products of the field components.

12.2 Justification

For weak fields or short distances ($h \ll 1$), the Taylor expansion converges rapidly and produces results comparable to a full RK integration with fewer arithmetic operations. The method is particularly efficient when the field integral can be precomputed or cached.

13 Linear Extrapolator

The `TrackLinearExtrapolator` provides field-free straight-line propagation:

$$x' = x + t_x\,\Delta z, \quad y' = y + t_y\,\Delta z, \quad t'_x = t_x, \quad t'_y = t_y, \tag{36}$$

with transport matrix:

$$F_{\text{linear}} = \begin{pmatrix} 1 & 0 & \Delta z & 0 & 0 \\ 0 & 1 & 0 & \Delta z & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}. \tag{37}$$

This is used in field-free regions (e.g., inside the VELO or beyond the magnet).

14 Parametrized Extrapolator

The `TrackParametrizedExtrapolator` approximates the propagation as a polynomial function of the input state:

$$s'_i = s_i + (q/p)\,\mathbf{c}_i^T\,\phi(\mathbf{s}), \tag{38}$$

where ϕ is a tensor-product polynomial basis of order $(0, 0, 2, 2, 4)$ in $(x, y, t_x, t_y, q/p)$, giving $(1)(1)(3)(3)(5) = 45$ basis functions.

14.1 Training

At initialisation, a grid of track states using **Chebyshev nodes** is propagated with `TrackRungeKutta-Extrapolator` between predefined detector planes (EndVelo → BeginUT, EndVelo → BeginT, etc.). The residual $\delta\mathbf{s} = \mathbf{s}_{\text{out}} - \mathbf{s}_{\text{linear}}$ is divided by q/p and fitted via LDLT least-squares decomposition, binned in 50×50 (x, y) cells.

15 Master Extrapolator

The `TrackMasterExtrapolator` is the production-level orchestrator. It does not solve the equations of motion itself but delegates to sub-extrapolators and handles material effects.

15.1 Dispatch Logic

The default `TrackDistanceExtraSelector` selects the sub-extrapolator based on propagation distance:

$$\text{extrapolator} = \begin{cases} \texttt{TrackParabolicExtrapolator} & \text{if } |\Delta z| < 100 \text{ mm}, \\ \texttt{TrackRungeKuttaExtrapolator} & \text{if } |\Delta z| \geq 100 \text{ mm}. \end{cases} \tag{39}$$

15.2 Justification

The 100 mm threshold balances accuracy and speed:

- For $|\Delta z| < 100$ mm, the magnetic field is nearly constant and the parabolic single-step method (1 field evaluation, $\mathcal{O}(\Delta z^3)$ error) is adequate. The position error is $\lesssim 10^{-4}$ mm for a 5 GeV track.
- For longer distances, the full adaptive RK is needed to maintain accuracy through regions of varying field strength.

15.3 Stepping and Material

1. Divide total Δz into steps of at most `MaxStepSize` (1000 mm).
2. For each step:
   (a) Validate: $|x|, |y| < 10$ m, $|t_x|, |t_y| < 5$.
   (b) Select and call the sub-extrapolator.
   (c) Accumulate transport matrix: $F_{\text{total}} = F_{\text{step}} \cdot F_{\text{prev}}$.
   (d) If material corrections are enabled, locate intersections with detector volumes and apply:
      - **Multiple scattering**: broadens angular distribution.
      - **Energy loss** (dE/dx): reduces momentum.
      - **Electron energy loss**: bremsstrahlung correction.

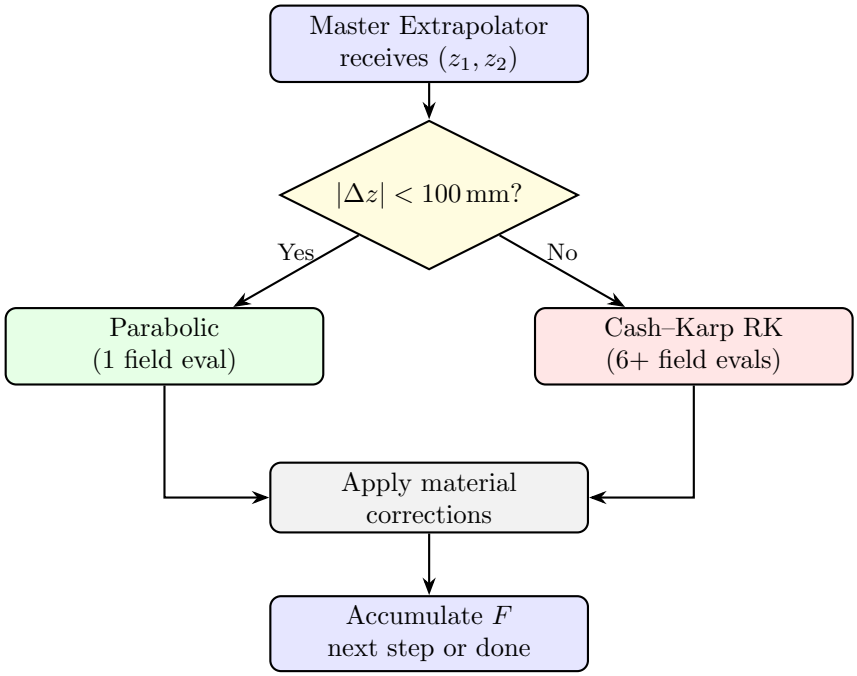

Figure 1: Master Extrapolator dispatch flow.

16 QMTest Regression Testing Pipeline

The LHCb CI system uses QMTest to validate extrapolator correctness. The pipeline is:

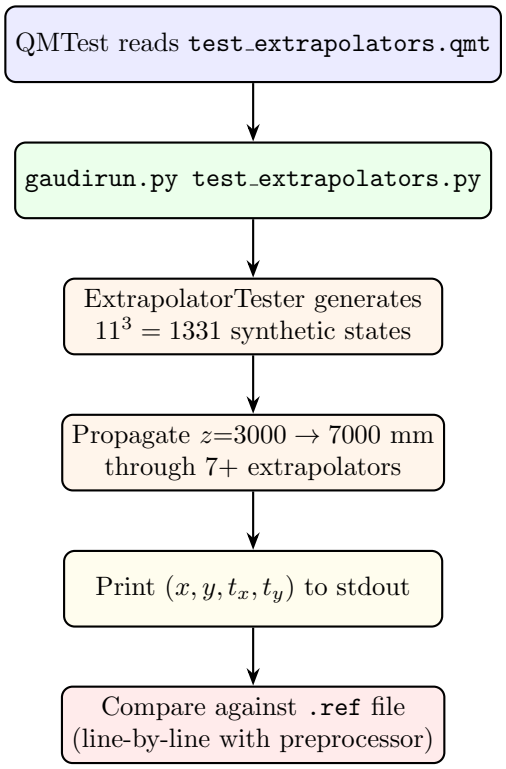

Figure 2: QMTest pipeline for extrapolator regression testing.

16.1 Test State Grid

The `ExtrapolatorTester` algorithm generates a Cartesian grid of initial track states:

- $q/p \in [-0.0004, +0.0004]$ MeV^{-1} (momenta $\gtrsim 2.5$ GeV)
- $t_x \in [-0.3, +0.3]$, $\quad t_y \in [-0.25, +0.25]$
- 11 bins per dimension $\Rightarrow$ 1331 test states
- Initial position: $x = t_x \cdot z_{\text{init}}$, $y = t_y \cdot z_{\text{init}}$ (tracks from origin)
- Propagation: $z_{\text{init}} = 3000 \rightarrow z_{\text{final}} = 7000$ mm

16.2 Extrapolators Under Test

The CI test (`test_extrapolators.py`) exercises:

1. Cash–Karp (Reference)
2. Bogacki–Shampine 3(2)
3. Verner 7(6)
4. Verner 9(8)
5. Tsitouras 5(4) (verbose mode)
6. Kisel
7. Herab

16.3 Validation

The stdout output is compared line-by-line against a reference file (`test_extrapolators.ref`, 10 778 lines). A `LineSkipper` preprocessor strips non-deterministic lines (timing, IOV, DD4Hep messages) before comparison. The test passes if and only if all remaining lines match exactly.

Platform-specific reference files exist for different architectures (e.g. `.ref.armv8`, `.ref.x86_64_v3-opt`) to account for floating-point differences.

17 SOA Benchmark Tester

For performance analysis (distinct from regression testing), the `TrackExtrapolatorTesterSOA` writes ROOT NTuples with quantitative data:

Table 3: NTuple columns written by `TrackExtrapolatorTesterSOA`.

Column	**Type**	**Description**
`qop, x_i, y_i, tx_i, ty_i`	float	Input state
`x_f, y_f, tx_f, ty_f`	float	Reference output
`dx, dy, dtx, dty`	float	Residuals
`time`	float	Propagation time (ns)
`success`	int	StatusCode
`field_cycles`, ...	int64	RDTSC sub-timers
`nsteps, nrejected`	int	Adaptive step stats

18 Summary Table of Extrapolators

Table 4: Complete inventory of LHCb track extrapolators.

Extrapolator	**Field evals**	**Order**	**Use case**
`TrackLinearExtrapolator`	0	1	Field-free regions
`TrackParabolicExtrapolator`	1	2	Short distances ($< 100\,\mathrm{mm}$)
`TrackRungeKuttaExtrapolator`	6/step	5(4)	General, production default
`TrackSTEPExtrapolator`	2/step	4	ATLAS-style RKN
`TrackHerabExtrapolator`	6/step	5(4)	HERA-B legacy
`TrackKiselExtrapolator`	3	1–3	CBM Taylor expansion
`TrackParametrizedExtrapolator`	0^*	poly	Plane-to-plane shortcuts
`TrackMasterExtrapolator`	—	—	Dispatcher + material

*Pre-trained at initialisation using RK.